TPO Algorithm Workflow

How to use the baseline, response and optimiser models on our dataset to answer the three Simulation Studio questions, with **discount as the only lever**.

Built from **Data/** as produced by **scripts/build_final_dataset.py** · 9 October 2026 · All numbers below are computed from the data when this PDF is generated.

1. The three questions this workflow answers

#	Business question	What the user gets back
Q1	**What if** I change discount by +5 points, or by 10% of its current value?	Revenue, incremental sales and profit at the new depth, with a P10–P90 range. No optimisation.
Q2	**What is the best discount**?	The recommended depth (e.g. "14% instead of today's 10%"), the gain, and its range.
Q3	**I want +X% revenue**. What discount gets me there, and if it can't, why not?	The required depth as a change from today (in points and relative %). If unreachable: the closest result and the constraint that blocked it.

Why discount only. Trade spend in our data is always **3.00%** of promoted units × list price. It never varies on its own, so no model can learn its effect. It stays a fixed cost inside profit and ROI. In Simulation Studio, the trade-spend slider works as a budget (how much of the scope can be funded), not as a response lever. Model it as a separate lever only once data exists where it varies independently of discount.

2. Our data, and what changed since the first review

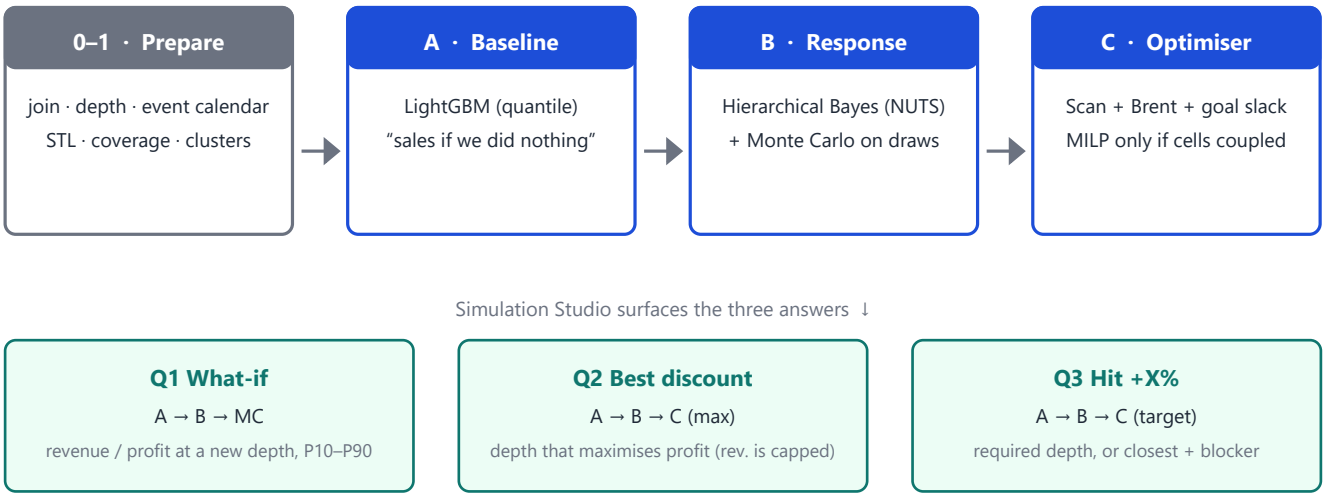
Fact	Value in Data/
Grain	**Weekly**: 139 weeks, 01 Jan 2024 to 24 Aug 2026. Model at week grain, not daily.
Size	325,260 rows · 36 products (9 brands) · 65 stores · 6 channels
Promoted rows	17.09% of rows
Discount depths seen	Regular 5 / 10 / 15% · festivals 20% (2024 "20% Discount") and 25% (2025–26 "Buy3Get1" = 1 free in 4). Small 9% / 11% clusters come from price rounding.
Events	New Year, Holi, Summer, Independence Day, Dussehra, Diwali. Festival offers sit in the festival week (or the month before, for Modern Trade and B2B, which book whole months).
Costs	Total_Cost = units × 50% of list (50% gross margin); Promotion_Cost = 3% × units × list

Concern raised in the chat	Status now	What it means for the workflow
Actual_Quantity = Base_Quantity, so there is no response to learn	**Resolved**. 0.00% of promoted rows have equal quantities. Base_Quantity is now the expected units without the promotion.	B has a real signal. Base_Quantity is a ready-made baseline target for A.
Only 3 depth levels	**Better**: 0, 5, 10, 15, 20 and 25%.	Enough to fit a curve. The optimiser must stay within 0–25%.
Festival effect confounded with discount	**Still true**: each festival runs at a single depth (20% or 25%), and regular offers never run in festival weeks.	Regular weeks give the curve shape. Events can only shift its level (see Step 3).
Trade spend fixed at 3%	**Still true**	Discount is the only lever (see box above).

Concern raised in the chat	Status now	What it means for the workflow
Quadratic curve bends down unrealistically	**Different in our data**: the response is convex. Deeper offers lift more than in proportion, by design of the dataset.	Revenue keeps rising up to the 25% cap. Profit is what peaks, so Q2 must optimise profit, or revenue with a margin floor.

3. The pipeline at a glance

Three modelling components, run in order. Of the original five algorithms, Monte Carlo is the last step of B, and goal programming with duals is the optimiser C with one extra variable.



Component	Algorithm	Runs	Python library
Step 1: explore	STL decomposition, event-study plots, k-means / HDBSCAN clustering	Once, as analysis	statsmodels, scikit-learn, hdbscan
A · Baseline	LightGBM, gradient-boosted trees with quantile loss (P10 / P50 / P90)	Always	lightgbm
B · Response	Hierarchical Bayesian regression sampled with NUTS (Hamiltonian Monte Carlo)	Always	pymc (or numpyro), arviz
B · Uncertainty	Monte Carlo: push each of ~1,000 posterior draws through the revenue formula	Always	numpy
C · Optimiser	Grid scan over depth, Brent's method for exact targets, goal-programming shortfall	Always	numpy, scipy.optimize.brentq
C · Coupled	MILP (branch-and-bound) + shadow prices from the LP re-solve with integers fixed	Only if cells share a budget	OR-Tools, PuLP or HiGHS

4. Step-by-step workflow

Step 0 · Build one modelling table

Goal	One row per product × store × week with every feature the models need.
Do	Join fact_sales to dim_product, dim_geo_store (channel, region, tier, retailer), dim_date and dim_promotion. Derive **depth d = 1 − Actual_Price / Base_Price**, **uplift u = Actual_Quantity / Base_Quantity − 1**, revenue = units × price, and **profit = Actual_Revenue − Total_Cost − Promotion_Cost**.
Event calendar	dim_date has no festival flags. Add them from the FESTIVALS dates in build_final_dataset.py, with a pre-event week and a post-event week to capture build-up and pull-forward dip. For Modern Trade and B2B, use the month leading into the festival.
Other features	Week of year, month, month-end and payday weeks, brand, size, channel, tier, region, season for Cough & Cold.
Where	app/tpo/loader.py already builds the joined frame. Reuse it; don't write a second loader.

Step 1 · Understand the data before fitting

Goal	Know which events and depths the data can support, and which products and stores behave alike.
Algorithms	**STL decomposition** of non-promoted weekly units (trend + season + residual). **Event-study plots**: units in the weeks around each festival, promoted and not, including the dip after. **k-means or HDBSCAN** on each product-store's seasonal profile, to define B's pooling groups.
Coverage check	Count promoted rows per event × depth (table below). An empty cell is a depth the optimiser must not recommend for that event.
Output	Pooling groups, a list of allowed depths per event, and a short note on the post-promotion dip.

Promotion type	5%	10%	15%	20%	25%
Regular	14,280	13,350	13,260	–	–
Seasonal	–	–	–	5,210	9,490

Promoted rows by type and depth, rounded to the nearest 5 points. Festival offers sit only at 20–25%, and regular offers only at 5–15%.

Step 2 · Component A: baseline (LightGBM)

Question	"What would we sell this week if we did nothing?"
Target	Weekly units with no promotion. **Option 1 (start here):** use Base_Quantity directly. The dataset already defines it as the expected units without the promotion. **Option 2:** train LightGBM on non-promoted rows only, then predict for promoted weeks. That also gives a baseline for weeks never promoted, which the Studio needs for future windows.
Features	Product, brand, size, store, channel, tier, region, week of year, event windows, lagged units (4 and 52 weeks).
Loss	Quantile loss at 0.1, 0.5 and 0.9. This trains three models and gives a P10–P90 baseline band.
Validate	Train 2024, test 2025. Check WAPE, check that ~80% of actuals fall inside P10–P90, and compare the P50 with Base_Quantity on promoted weeks (they should agree closely).
Output	baseline_units[product, store, week] with P10 / P50 / P90.

Step 3 · Component B: response model (hierarchical Bayes + NUTS)

Question	"How much extra volume does each discount depth add, per brand, channel and event, and how sure are we?"
Model	$\log(1 + u) = \beta_1[g] \cdot d + \beta_2[g] \cdot d^2 + \gamma[\text{event}] + \varepsilon$. Here g is the pooling group (brand × channel, or the clusters from Step 1). The curve passes through zero lift at zero discount.
Hierarchy	$\beta[g] \sim \text{Normal}(\beta_{\text{brand}}, \sigma)$ and $\beta_{\text{brand}} \sim \text{Normal}(\beta_{\text{global}}, \tau)$. $\gamma[\text{event}] \sim \text{Normal}(\gamma_{\text{festival type}}, \sigma_e)$, so Dussehra borrows from Diwali but can differ.
Confounding rule	Regular weeks (5/10/15%) identify the curve **shape** β. Festival weeks identify only the event **level** γ, because each festival has one depth. Do not let β vary by event. The data cannot support it.
Grain	Fit on product × channel × week totals, not product × store rows. That is more stable and far faster to sample.
Fit	PyMC with the NUTS sampler: 4 chains × 1,000 draws. Diagnostics: R-hat < 1.01, zero divergences, effective sample size > 400.
Backtest	Hold out Diwali 2025 and Dussehra 2025, refit, and compare predicted with actual uplift.
Output	~1,000 posterior draws of $(\beta_1, \beta_2, \gamma)$ for every group and event.

Step 4 · Monte Carlo (end of B)

Do	For each draw k and each candidate depth d: $units_k$ = baseline × (1 + $u_k(d)$), $revenue_k$ = $units_k$ × list × (1 − d), $profit_k$ = $units_k$ × list × (1 − d − 0.50 − 0.03).
Output	P10 / P50 / P90 of revenue, incremental sales, profit and ROI at every depth. This is a few lines of numpy, not a separate model.

Step 5 · Component C: optimiser

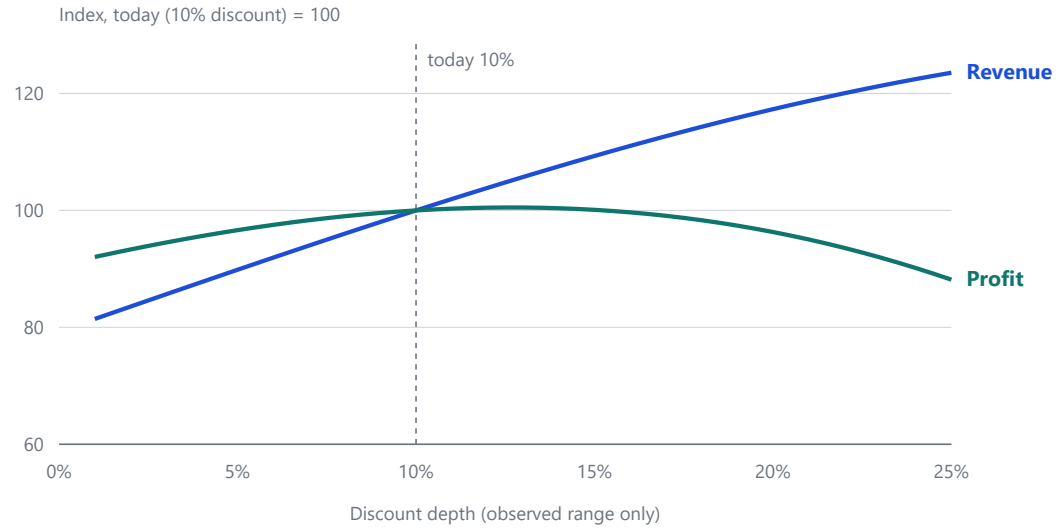
Scan	For each cell, evaluate depths from 0% to 25% in 0.5-point steps, across all draws. Never go beyond the deepest depth observed for that event (Step 1).
Q1	Read the P10 / P50 / P90 at the new depth. No optimisation needed.
Q2	Pick the depth with the highest median **profit**, or highest revenue subject to a margin floor and the depth cap.
Q3	Goal programming: minimise the shortfall s in revenue ≥ (1 + X%) × today − s. If s = 0 at some depth, use Brent's method to find the exact depth (the shallowest, because it costs least margin). If s > 0 at every depth, report the closest result and the binding constraint.
MILP	Only when cells share a rule, e.g. the Studio's trade-spend budget across a whole scope, or a brand-level margin floor. Then solve the MILP and fix the integer choices. Re-solve as an LP and read the shadow prices to explain which constraint blocked the target.

Step 6 · Plug into Simulation Studio

Do	Swap the Studio's single pooled curve for B's draws. Keep the Studio's promise: it never computes a KPI of its own. It decides which rows exist and what quantity each carries, then hands them to app/tpo/aggregate.py (Revenue, Trade Spend, Incremental Sales, ROI).
Note	Simulation Studio and Decision Center are frozen. Any user-visible change needs sign-off first.

5. The three questions on our numbers

Regular weeks, pooled across all products. The curve was fitted to the 40,890 regular promoted rows: **$\log(1 + u) = 3.67 \cdot d - 2.98 \cdot d^2$**. A brand × channel fit from Step 3 will differ per cell; this is the shape of the method. Example cell: 1,000 baseline units, list price ₹100, today at 10% off. Ranges are the P10–P90 of 1,000 bootstrap refits, resampling product-store pairs. They are a stand-in for B's posterior draws, and show the uncertainty of the curve itself. A single store-week also scatters around the curve, by about -0.23 to +0.27 on the log scale.



Lines start at 1% because any promotion adds the 3% trade-spend cost. Revenue rises across the whole observed range, because the response is convex. Profit peaks near **12.50%**. This is why Q2 must optimise profit, or revenue under a margin floor.

Q1 · What-if

Scenario	Depth	Units	Revenue	vs today	Profit
No promotion	0.00%	1,000	₹1,00,000	-20.68%	₹50,000
Today	10.00%	1,401	₹1,26,075	+0.00%	₹51,831
+10% relative (10 → 11%)	11.00%	1,444	₹1,28,525	+1.94%	₹51,988
+5 points (10 → 15%)	15.00%	1,621	₹1,37,812	+9.31%	₹51,882
Deepest observed (25%)	25.00%	2,077	₹1,55,767	+23.55%	₹45,692

Revenue range at 15% (bootstrap P10–P90): ₹1,36,675 to ₹1,38,952.

Q2 · Best discount

Objective	Answer	Change from today
Maximise revenue	25.00%. It hits the observed cap, so the true peak lies outside our data.	+15.00 pts · +150.00% relative
Maximise profit (recommended)	**12.50%**, profit ₹52,090	+2.50 pts · +25.00% relative

Q3 · Hit a revenue target (vs today's revenue)

Target	Required depth	Change from today	Notes
+5%	**12.61%** (range 12.53% to 12.71%)	+2.61 pts · +26.13% relative	Shallowest depth that reaches the target
+20%	**21.97%**	+11.97 pts · +119.67% relative	Profit falls to ₹48,534. Flag this to the user.
+40%	Not reachable	Closest: +23.55% at 25%	Shortfall 16.45%. Blocker: no evidence beyond 25% discount.

Always report the change both ways. 10% → 12% is **+2 points** and also **+20% relative**. Business users mix these up, so the Studio should show both.

Events: why Diwali and Dussehra need their own term

For each festival: observed (1 + uplift) divided by what the regular-week curve predicts at the same depth. This median ratio is a raw preview of the γ[event] term B will estimate properly.

Event	Promoted rows	Depths run	Lift vs a regular week at the same depth
Diwali Special	1,290	20%, 25%	× 1.49
Holi Special Deal	3,015	20%, 25%	× 1.28
New Year Savings	2,265	20%, 25%	× 1.19
Dussehra Deal	2,260	20%, 25%	× 1.14
Independence Offer	2,895	20%, 25%	× 1.10
Summer Special	2,975	20%, 25%	× 1.04

Read it as: the same discount sells this many times more in that festival week than in a normal week. Each festival ran at one depth per year (20% in 2024, 25% from 2025), so depth is tangled with the year. Treat this as a level shift on the regular curve, not a separate curve.

6. Simulation Studio: today vs. after this workflow

Piece	Studio today (app/tpo/studio.py)	After the workflow
Baseline	Per product-channel non-promotional baseline (the rule in aggregate._volume)	A: LightGBM P50 with a P10–P90 band. Also covers future windows.
Response curve	One weighted least-squares fit, log(1+u) = b1·d + b2·d^2, for the scope. Falls back to the whole dataset, then to config rules.	B: hierarchical Bayes per brand × channel with an event term. Thin cells borrow strength instead of falling back to one global curve.
Range shown	P10–P90 of the residuals	Monte Carlo over ~1,000 posterior draws (parameter + noise uncertainty)
Discount lever	Continuous, bounded by observed depths	Same bound, per event (from the Step 1 coverage check)
Days lever	N/7 weeks, fade and post-promo dip estimated from data	Unchanged
Trade-spend lever	Budget → coverage = min(1, S / full cost)	Unchanged. If the budget is shared across cells, C becomes a MILP (knapsack).
Best / target answers	–	C: scan + Brent + shortfall, giving Q2 and Q3 answers with the blocker named

7. Decisions to settle before building

- **What is +X% measured against?** Today's revenue (used above), the no-promotion baseline, or last year's same period? Each gives a different target number.
- **Objective for Q2:** profit (recommended), or revenue with a margin floor. Pure revenue just hits the depth cap.
- **Depth cap:** 0–25% overall, and per event only the depths actually run.
- **Pooling group for B:** brand × channel (simple) or the Step 1 clusters (better when cells are sparse).
- **Buy3Get1:** treated as a 25% price cut, which is how the fact table prices it. If the business sees it as a different mechanic, add a mechanic term in B.
- **Trade spend:** stays a fixed 3% cost until a test varies it independently of discount.

8. Build order

#	Task	Done when
1	Modelling table + event calendar (Step 0)	One frame, every feature, reconciles to Insights Hub totals
2	Exploration (Step 1)	Coverage table, pooling groups and dip note signed off
3	Baseline A (Step 2)	2025 backtest WAPE and ~80% P10–P90 coverage
4	Response B + Monte Carlo (Steps 3–4)	R-hat < 1.01, no divergences, festival hold-out within band
5	Optimiser C for Q1–Q3 (Step 5)	Answers match a brute-force scan; blockers named
6	Studio integration (Step 6)	Approved change to the frozen module; existing tests pass
7	MILP	Only if a shared budget or margin rule is required

9. Learning path for a beginner

Work through these in order. One link is verified; the other items are exact titles or channels to search on YouTube or Google. None of the links have been invented.

Step	Topic (component)	Watch / read
1	Gradient boosting & LightGBM (A)	YouTube: "StatQuest Gradient Boost Part 1" (Josh Starmer), then parts 2–4. Search: "LightGBM explained leaf-wise histogram". Search: "quantile regression pinball loss" (for P10–P90).
2	Bayesian thinking & partial pooling (B)	YouTube: "Statistical Rethinking" by Richard McElreath, the multilevel-model lectures (free). Search: "PyMC hierarchical model partial pooling". Talk: "Hierarchical Models in MMM: Can Structure beat data size?" (PyCon DE & PyData 2026).
3	NUTS / Hamiltonian Monte Carlo (B)	YouTube: "Ben Lambert Hamiltonian Monte Carlo". YouTube: "No-U-Turn Sampler Matt Hoffman Google TechTalk". You only need the idea, not the maths.
4	Monte Carlo simulation (end of B)	YouTube (verified): "A Simple Solution for Really Hard Problems: Monte Carlo Simulation" https://www.youtube.com/watch?v=slbZ-SLplgg
5	Optimisation: search, Brent, MILP (C)	Search: "Brent's method root finding". YouTube: "Integer Programming: How to Use the Branch and Bound Method", then "MIP Solving: Branch and Bound". Search: "piecewise linear approximation MILP SOS2".
6	Why a target failed (C)	Search: "Lagrange multipliers shadow price intuition". Wikipedia: "Shadow price". Search: "goal programming deviation variables operations research".
7	Data exploration (Step 1)	Search: "STL decomposition time series explained", "event study analysis marketing", "k-means clustering StatQuest".

10. Glossary

Term	Plain meaning
Depth (d)	How much the price is cut: 1 − Actual_Price / Base_Price. 10% off means d = 0.10.
Uplift (u)	Extra units from the promotion, relative to the baseline. u = 0.40 means 40% more units.
Baseline	Units we would have sold with no promotion that week.
Points vs relative	10% → 12% is +2 points (absolute) and +20% relative. Always show both.
Partial pooling	Small groups borrow information from similar groups, but keep their own estimate.
Posterior draws	~1,000 plausible versions of the model's numbers. Their spread is the uncertainty.
P10 / P50 / P90	There is a 10% chance the result falls below P10 and a 90% chance it falls below P90. P50 is the middle.
Shortfall (s)	How far the best achievable result is from the target. s = 0 means the target is reachable.
Shadow price	How much the result would improve if a constraint (cap, budget, floor) were relaxed by one unit.
R-hat	Sampler health check. Close to 1.00 means the chains agree.